

第34章 TIM—电容按键检测

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

前面章节我们讲解了基本定时器和高级控制定时器功能，这一章我们将介绍定时器输入捕获一个应用实例，帮助我们更加深入理解定时器。

34.1 电容按键原理

电容器(简称为电容)就是可以容纳电荷的器件，两个金属块中间隔一层绝缘体就可以构成一个最简单的电容。如图 34-1(俯视图)，有两个金属片，之间有一个绝缘介质，这样就构成了一个电容。这样一个电容在电路板上非常容易实现，一般设计四周的铜片与电路板地信号连通，这样一种结构就是电容按键的模型。当电路板形状固定之后，该电容的容量也是相对稳定的。

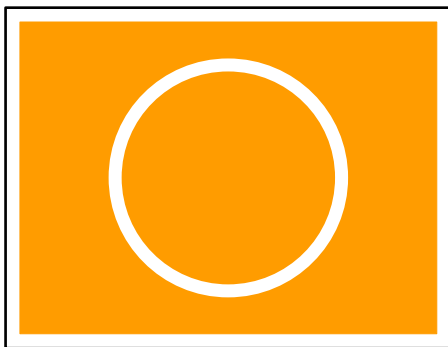


图 34-1 片状电容器

电路板制作时都会在表面上覆盖一层绝缘层，用于防腐蚀和绝缘，所以实际电路板设计情况如图 34-2。电路板最上层是绝缘材料，下面一层是导电铜箔，我们根据电路走线情况设计决定铜箔的形状，再下面一层一般是 FR-4 板材。金属感应片与地信号之间有绝缘材料隔着，整个可以等效为一个电容 C_x 。一般在设计时候，把金属感应片设计成方便手指触摸大小。

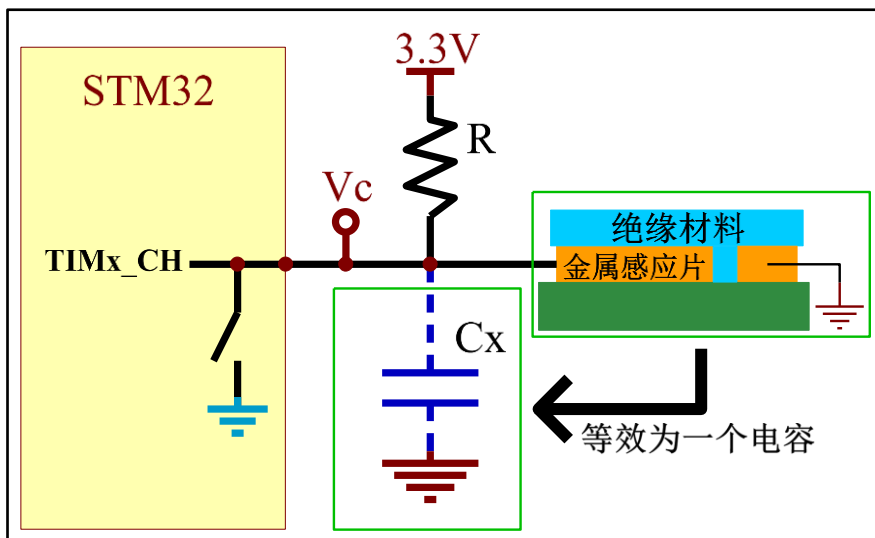


图 34-2 无手指触摸情况

在电路板未上电时，可以认为电容 C_x 是没有电荷的，在上电时，在电阻作用下，电容 C_x 就会有一个充电过程，直到电容充满，即 V_c 电压值为 3.3V，这个充电过程的时间长短受到电阻 R 阻值和电容 C_x 容值的直接影响。但是在我们选择合适电阻 R 并焊接固定到电路板上后，这个充电时间就基本上不会变了，因为此时电阻 R 已经是固定的，电容 C_x 在无外界明显干扰情况下基本上也是保持不变的。

现在，我们来看看当我们用手指触摸时会是怎样一个情况？如图 34-3，当我们用手指触摸时，金属感应片除了与地信号形成一个等效电容 C_x 外，还会与手指形成一个 C_s 等效电容。

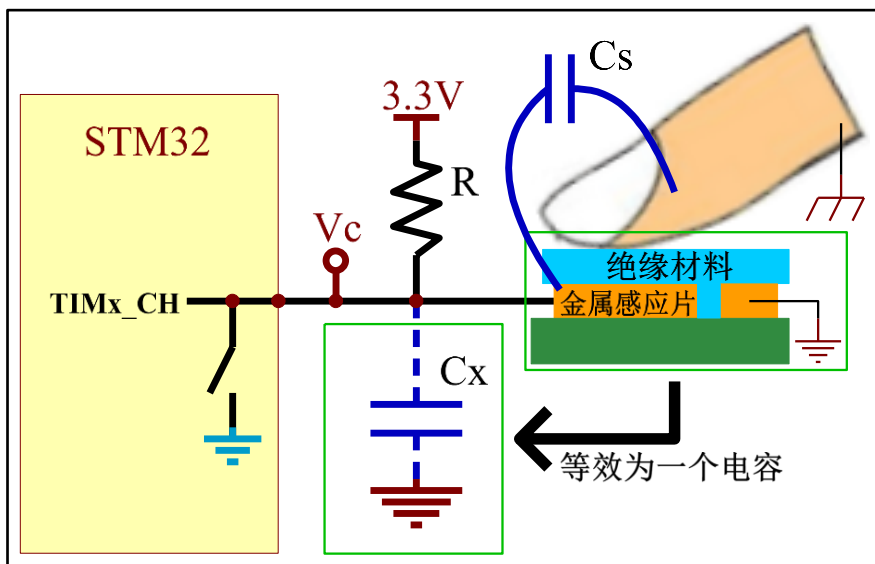


图 34-3 有手指触摸情况

此时整个电容按键可以容纳的电荷数量就比没有手指触摸时要多了，可以看成是 C_x 和 C_s 叠加的效果。在相同的电阻 R 情况下，因为电容容值增大了，导致需要更长的充电时间。也就是这个充电时间变长使得我们区分有无手指触摸，也就是电容按键是否被按下。

现在最主要的任务就是测量充电时间。充电过程可以看出是一个信号从低电平变成高电平的过程，现在就是要求出这个变化过程的时间，这样的一个命题与上一章讲解高级控制定时器的输入捕获功能非常吻合。我们可以利用定时器输入捕获功能计算充电时间，即设置 TIMx_CH 为定时器输入捕获模式通道。这样先测量得到无触摸时的充电时间作为比较基准，然后再定时循环测量充电时间与无触摸时的充电时间作比较，如果超过一定的阈值就认为是有手指触摸。

图 34-4 为 V_c 跟随时间变化情况，可以看出在无触摸情况下，电压变化较快；而在有触摸时，总的电容量增大了，电压变化缓慢一些。

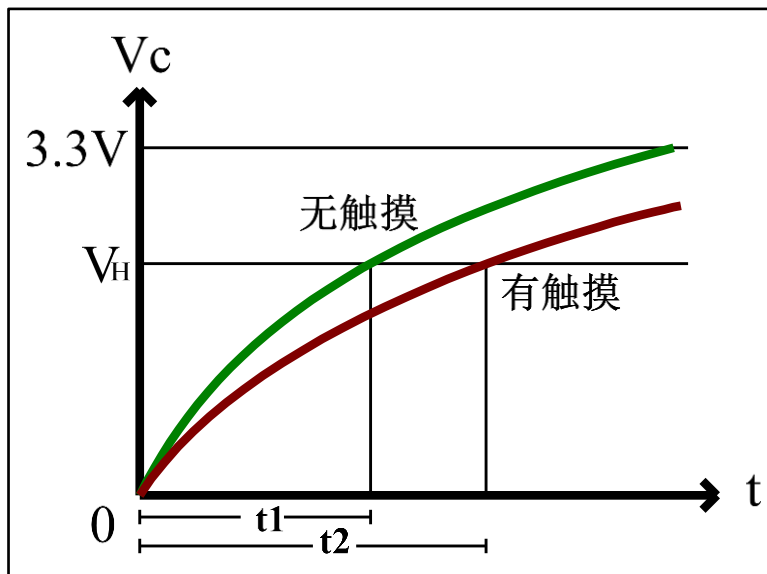


图 34-4 V_c 电压与充电时间关系

为测量充电时间，我们需要设置定时器输入捕获功能为上升沿触发，图 34-4 中 V_H 就是被触发上升沿的电压值，也是 STM32 认为是高电平的最低电压值，大约为 1.8V。 t_1 和 t_2 可以通过定时器捕获/比较寄存器获取得到。

不过，在测量充电时间之前，我们必须想办法制作这个充电过程。之前的分析是在电路板上电时会有充电过程，现在我们要要求在程序运行中循环检测按键，所以必须可以控制充电过程的生成。我们可以控制 TIMx_CH 引脚作为普通的 GPIO 使用，使其输出一小段时间的低电平，为电容 C_x 放电，即 V_c 为 0V。当我们重新配置 TIMx_CH 为输入捕获时电容 C_x 在电阻 R 的作用下就可以产生充电过程。

34.2 电容按键检测实验

电容按键不需要任何外部机械部件，使用方便，成本低，很容易制成与周围环境相密封的键盘，以起到防潮防湿的作用。电容按键优势突出使得越来越多电子产品使用它代替传统的机械按键。

本实验实现电容按键状态检测方法，提供一个编程实例。

34.2.1 硬件设计

开发板板载一个电容按键，原理图设计参考图 34-5。

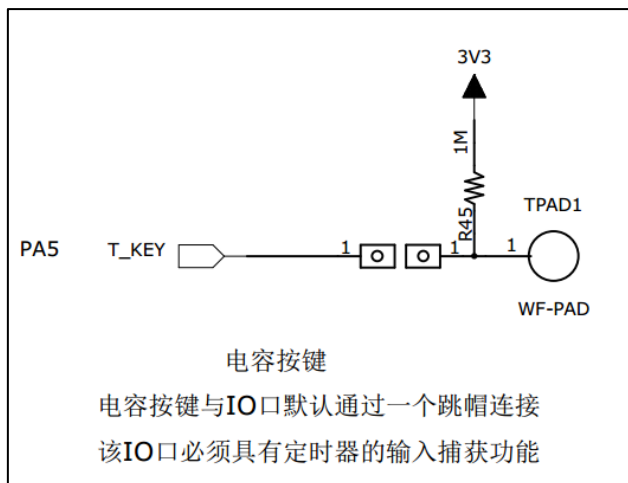


图 34-5 电容按键电路设计

开发板中标示“电容按键”的地方就是电容按键实体，实际上就是一块铜皮，然后在电路板上通过一根导线连接至定时器通道引脚，并带一个 3.3V 的上拉电阻，电阻的大小和电容按键的大小决定了充电的时间长度。默认上电情况下电容按键会被 3.3V 通过电阻 R45 充满电。实验还用到调试串口和蜂鸣器功能，用来打印输入捕获信息和指示按键状态，这两个模块电路可参考之前相关章节。

34.2.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：bsp_touchpad.c 和 bsp_touchpad.h 文件用来存放电容按键检测相关函数和宏定义。

1. 编程要点

- (1) 编写定时器输入捕获相关函数
- (2) 测量电容按键空载的充电时间 T1
- (3) 测量电容按键有手触摸的充电时间 T2
- (4) 只需要比较 T2 与 T1 的时间即可检测出按键是否有手指触摸

2. 软件分析

电容按键宏定义

代码清单 34-1 电容按键宏定义

```
1 /***** TPAD 引脚配置参数定义 *****/
2 #define TPAD_TIMx TIM2
3 #define TPAD_TIM_CLK RCC_APB1Periph_TIM2
4
5 #define TPAD_TIM_Channel_X TIM_Channel_1
6 #define TPAD_TIM_IT_CCx TIM_IT_CC1
7 #define TPAD_TIM_GetCaptureX TIM_GetCapture1
8 #define TPAD_TIM_PERIOD 0xFFFF
9 #define TPAD_TIM_PSC (21-1)
10
```

```
11 //定时器最大计数值
12 #define TPAD_ARR_MAX_VAL 0xFFFF
13
14 // TIM 输入捕获通道 GPIO 相关宏定义
15 #define TPAD_TIM_GPIO_CLK      RCC_AHB1Periph_GPIOA
16 #define TPAD_TIM_CH_PORT      GPIOA
17 #define TPAD_TIM_CH_PIN      GPIO_Pin_5
18 #define TPAD_TIM_AF      GPIO_AF_TIM2
19 #define TPAD_TIM_SOURCE      GPIO_PinSource5
20
21 // 电容按键被按下的时候门限值，需要根据不同的硬件实际测试，
22 // 减小这个门限值可以提高响应速度
23 #define TPAD_GATE_VAL      30
24
25 // 电容按键空载的时候的最大和最小的充电时间，不同的硬件不一样，
26 // F407 霸天虎稳定在 32
27 #define TPAD_DEFAULT_VAL_MIN      30
28 #define TPAD_DEFAULT_VAL_MAX      35
29
30 #define TPAD_ON      1
31 #define TPAD_OFF      0
```

有关宏的具体含义配套注释阅读即可。定时器计数器的时钟分频因子，我们通过宏 TPAD_TIM_PSC 默认配置为 20，则计数器的计数时间为 $(20+1)/84M = 1\mu s$ ，自动重装载寄存器 ARR 的值通过 TPAD_TIM_Period 默认配置为 0xFFFF，即 65535。所以，计数器在不发生溢出的情况下，能计数的最长时间为 65.535ms。而一般电容按键的充电时间都是 us 级别，所以定时器的这个配置足以。在 F407-霸天虎中，电容按键的空载充电时间为 32us，当有手指轻微触摸电容按键的时候，充电时间会上升为 65us 以上，如果手指的触摸面积更大的话，充电时间也就相应的会变的更长。在编写代码的时候，我们只需要比较这两个充电时间即可知道是否触摸按键被按下。

电容按键 GPIO 配置

代码清单 34-2 电容按键 GPIO 配置

```
1 static void TPAD_TIM_GPIO_Config(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
4
5     // 使能通道引脚时钟
6     RCC_AHB1PeriphClockCmd(TPAD_TIM_GPIO_CLK, ENABLE);
7     // 指定引脚复用
8     GPIO_PinAFConfig(TPAD_TIM_CH_PORT,TPAD_TIM_SOURCE,TPAD_TIM_AF);
9
10    // 端口配置
11    GPIO_InitStructure.GPIO_Pin = TPAD_TIM_CH_PIN;
12    // 复用功能
13    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
14    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
15    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
16    //不带上下拉
17    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
18    GPIO_Init(TPAD_TIM_CH_PORT, &GPIO_InitStructure);
19 }
```

TPAD_TIM_GPIO_Config()函数初始化了定时器用到的相关的 GPIO，当使用不同的 GPIO 的时候，只需要修改头文件里面的宏定义即可，而不需要修改这个函数。

电容按键定时器模式配置

代码清单 34-3 电容按键定时器模式配置

```
1 static void TPAD_TIM_Mode_Config(void)
2 {
3     TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
4     TIM_ICInitTypeDef  TIM_ICInitStructure;
5
6     // 使能 TIM 时钟, TIMx_CLK = 2 * PCLK1 = 84M
7     RCC_APB1PeriphClockCmd(TPAD_TIM_CLK, ENABLE);
8     /*-----时基结构体初始化-----*/
9     // 自动重装载寄存器的值, 累计 TIM_Period+1 个频率后产生一个更新或者中断
10    TIM_TimeBaseStructure.TIM_Period = TPAD_TIM_PERIOD;
11    // 驱动 CNT 计数器的时钟 = TIMx_CLK / (psc+1)
12    TIM_TimeBaseStructure.TIM_Prescaler = TPAD_TIM_PSC;
13    // 时钟分频因子, 配置死区时间时需要用到, 这里不需要
14    TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
15    // 计数器计数模式
16    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
17    // 初始化定时器
18    TIM_TimeBaseInit(TPAD_TIMx, &TIM_TimeBaseStructure);
19    /*-----输入捕获结构体初始化-----*/
20    // 配置输入捕获的通道, 需要根据具体的 GPIO 来配置
21    TIM_ICInitStructure.TIM_Channel = TPAD_TIM_Channel_X;
22    // 输入捕获信号的极性配置
23    TIM_ICInitStructure.TIM_ICPolarity = TIM_ICPolarity_Rising;
24    // 输入通道和捕获通道的映射关系, 有直连和非直连两种
25    TIM_ICInitStructure.TIM_ICSelection = TIM_ICSelection_DirectTI;
26    // 配置输入分频, 不分频
27    TIM_ICInitStructure.TIM_ICPrescaler = TIM_ICPSC_DIV1;
28    // 配置输入滤波器 不滤波
29    TIM_ICInitStructure.TIM_ICFilter = 0x00;
30    // 定时器输入捕获初始化
31    TIM_ICInit(TPAD_TIMx, &TIM_ICInitStructure);
32
33    // 使能计数器
34    TIM_Cmd ( TPAD_TIMx, ENABLE );
35 }
```

TPAD_TIM_Mode_Config()函数中初始化了两个结构体, 有关这两个结构体成员的具体含义可参考“定时器初始化结构体详解”小节, 剩下的程序参考注释阅读即可。有个地方要注意的是捕获信号的极性配置, 需要配置为上升沿。因为电容按键在放电之后再充电的时候是一个电平又低到高的过程。

电容按键复位

代码清单 34-4 电容按键复位

```
1 /**
2  * @brief  复位电容按键, 放电, 重新充电
3  * @param  无
4  * @retval 无
5  * 说明:
6  * 开发板上电之后, 电容按键默认已经充满了电, 要想测得电容按键的充电时间
7  * 就必须先把电容按键的电放掉, 方法为让接电容按键的 IO 输出低电平即可
8  * 放电完毕之后, 再把连接电容按键的 IO 配置为输入, 然后通过输入捕获的方法
9  * 测量电容按键的充电时间, 这个充电时间是没有手指触摸的情况下的充电时间
10 * 而且这个空载的充电时间非常稳定, 因为电路板的硬件已经确定了
11 *
12 * 当有手指触摸的情况下, 充电时间会变长, 我们只需要对比这两个时间就可以
13 * 知道电容按键是否有手指触摸
14 */
```

```
15 static void TPAD_Reset(void)
16 {
17     GPIO_InitTypeDef GPIO_InitStructure;
18
19     // 使能通道引脚时钟
20     RCC_AHB1PeriphClockCmd(TPAD_TIM_GPIO_CLK, ENABLE);
21
22     //配置引脚为普通推挽输出
23     GPIO_InitStructure.GPIO_Pin = TPAD_TIM_CH_PIN;
24     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
25     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
26     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
27     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_DOWN;
28     GPIO_Init(TPAD_TIM_CH_PORT, &GPIO_InitStructure);
29
30     // 连接 TPAD 的 IO 配置为输出, 然后输出低电平, 延时一会, 确保电容按键放电完毕
31     GPIO_ResetBits ( TPAD_TIM_CH_PORT, TPAD_TIM_CH_PIN );
32     // 放电是很快的, 一般是 us 级别
33     //Delay_ms(5);
34     SysTick_Delay_Ms(5);
35
36     // 连接 TPAD 的 IO 配置为输入, 用于输入捕获
37     GPIO_InitStructure.GPIO_Pin = TPAD_TIM_CH_PIN;
38     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
39     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
40     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
41     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
42     GPIO_Init(TPAD_TIM_CH_PORT, &GPIO_InitStructure);
43 }
```

开发板上电之后, 电容按键默认已经充满了电, 要想测得电容按键的充电时间 就必须先把电容按键的电放掉, 方法为让接电容按键的 IO 输出低电平即可, 这个放电的时间一般都是 us 级别, 我们可以稍微延时以下即可。放电完毕之后, 再把连接电容按键的 IO 配置为输入, 然后通过输入捕获的方法测量电容按键的充电时间, 这个充电时间 T1 是没有手指触摸的情况下的充电时间, 而且这个空载的充电时间非常稳定, 因为电路板的硬件已经确定了。当有手指触摸的情况下, 相当于电容变大, 充电时间 T2 会变长, 我们只需要对比这两个时间就可以 知道电容按键是否有手指触摸。

电容按键初始化

代码清单 34-5 电容按键初始化

```
1 /**
2  * @brief  初始化触摸按键, 获得空载的时候触摸按键的充电时间
3  * @param  无
4  * @retval 0: 成功, 1: 失败
5  * @note   空载值一般很稳定, 由硬件电路决定, 该函数只需要调用一次即可
6  *         而且这个空载的充电时间每个硬件都不一样, 最好实际测试下
7  */
8 uint8_t TPAD_Init(void)
9 {
10     uint16_t temp;
11
12     // 电容按键用到的输入捕获的 IO 和捕获模式参数初始化
13     TPAD_TIM_Init();
14
15     temp = TPAD_Get_Val();
16
17     // 调试的时候可以把捕获的值打印出来, 看看默认的充电时间是多少
18     printf("电容按键默认充电时间为: %d us\n", temp);
19
20     // 电容按键空载的充电时间非常稳定, 不同的硬件充电时间不一样
```



```
21 // 需要实际测试所得，F407-霸天虎上的电容按键空载充电时间稳定在 32us
22 // 如果你觉得单次测量不准确，你可以多次测量然后取个平均值
23 if ( (TPAD_DEFAULT_VAL_MIN<temp) && (temp<TPAD_DEFAULT_VAL_MAX) ) {
24     tpad_default_val = temp;
25     return 0; // 成功
26 } else {
27     return 1; // 失败
28 }
29 }
```

TPAD_Init() 函数用来获取电容按键空载的充电时间，当获取到之后，把值存在 tpad_default_val 这个全局变量当中。这个空载的充电时间不同的硬件是不一样的，需要实际测试，在调试的过程中，可把捕获到的值打印出来看看。F407-霸天虎开发板上这个值稳定在 32us。

在 TPAD_Init() 函数中，我们是通过调用 TPAD_Get_Val() 函数来获取电容按键的充电时间的。当电容按键从 0 开始充电到 STM32 能够识别的高电平时，定时器则发生捕获，此时计数器的值会被锁存到输入捕获寄存器，我们只需要读取输入捕获寄存器的值，就可以算出这个充电的时间。通过 TPAD_Get_Val() 这个函数，我们可以测出电容按键的空载充电时间 T1 和有手触摸的情况下的充电时间 T2。

获取定时器输入捕获值

代码清单 34-6 获取定时器输入捕获值

```
1 /**
2  * @brief 获取定时器捕获值
3  * @param 无
4  * @retval 定时器捕获值。如果超时，则直接返回定时器的计数值。
5  */
6 uint16_t TPAD_Get_Val(void)
7 {
8     // 每次捕获的时候，必须先复位放电
9     TPAD_Reset();
10
11     // 当电容按键复位放电之后，计数器清 0 开始计数
12     TIM_SetCounter(TPAD_TIMx, 0);
13
14     // 清除相关的标志位
15     TIM_ClearITPendingBit(TPAD_TIMx, TPAD_TIM_IT_CCx|TIM_IT_Update);
16
17     // 等待捕获上升沿，当电容按键充电到 1.8v 左右的时候，就会被认为是上升沿
18     while (TIM_GetFlagStatus (TPAD_TIMx, TPAD_TIM_IT_CCx) == RESET) {
19         // 如果超时了，直接返回 CNT 的值
20         // 一般充电时间都是在 ms 级别以内，很少会超过定时器的最大计数值
21         if (TIM_GetCounter(TPAD_TIMx)>TPAD_ARR_MAX_VAL-500) {
22             return TIM_GetCounter(TPAD_TIMx);
23         }
24     }
25
26     // 获取捕获比较寄存器的值
27     return TPAD_TIM_GetCaptureX(TPAD_TIMx);
28 }
```

获取最大输入捕获值

代码清单 34-7 获取最大捕获值

```
1 /**
```



```
2  * @brief  读取若干次定时器捕获值，并返回最大值。
3  * @param  num : 读取次数
4  * @retval 读取到的最大定时器捕获值
5  */
6  uint16_t TPAD_Get_MaxVal( uint8_t num )
7  {
8      uint16_t temp=0, res=0;
9
10     while (num-->0) {
11         temp = TPAD_Get_Val();
12         if ( temp > res )
13             res = temp;
14     }
15     return res;
16 }
```

该函数接收一个参数，用来指定获取电容按键捕获值的循环次数，函数的返回值则为 num 次发生捕获中最大的捕获值。当我们用手指触摸电容按键的时候，常常会有干扰或者是误触发，所以我们一般选取最大的值为有效值。

电容按键状态扫描

代码清单 34-8 电容按键状态扫描

```
1  /**
2   * @brief  按键扫描函数
3   * @param  无
4   * @retval 1: 按键有效, 0: 按键无效
5   */
6  uint8_t TPAD_Scan(void)
7  {
8      // keyen: 按键检测使能标志
9      // 0: 可以开始检测
10     // >0: 还不能开始检测，表示按键一直被按下
11     // 注意: keytn 这个变量由 static
12     // 修饰，相当于一个全局变量，但是因为是在函数内部定义，
13     // 所以是相当于这个函数的全局变量，每次修改之前保留的是上一次的值
14     static uint8_t keyen=0;
15
16     uint8_t res=0, sample=3;
17     uint16_t scan_val;
18
19     // 根据 sample 值采样多次，并取最大值，小的一般是干扰或者是误触摸
20     scan_val = TPAD_Get_MaxVal(sample);
21
22     // 当扫描的值大于空载值加上默认的门限值之后，表示按键按下
23     // 这个 TPAD_GATE_VAL 根据硬件决定，需要实际测试
24     if (scan_val > (tpad_default_val+TPAD_GATE_VAL)) {
25         // 再次检测，类似于机械按键的去抖
26         scan_val = TPAD_Get_MaxVal(sample);
27         if ( ( keyen ==0)&&(scan_val > (tpad_default_val+TPAD_GATE_VAL)) )
28             res = 1; // 有效的按键
29
30         // 如果按键一直被按下，keyen 的值会一直在 keyen 的初始值和 keyen-
31         // 1 之间循环，永远不会等于 0
32         keyen = 2;
33     }
34
35     // 当按键没有被按下或者 keyen>0 时，会执行 keyen--
36     if ( keyen > 0)
37         keyen--;
38
39     return res;
40 }
```

按键扫描函数不断的检测充电时间，当大于 `tpad_default_val+TPAD_GATE_VAL` 时，表示按键被按下，其中 `TPAD_GATE_VAL` 是一个宏，具体多大需要实际测试。具体的我们可以通过调用 `TPAD_Get_Val()` 函数来测试按键有手触摸的情况下的充电值，然后再减去 `tpad_default_val` 的值就可以得到 `TPAD_GATE_VAL`，当减小这个门限值的时候可以提提高按键的灵敏度。

在按键扫描函数中，我们引入了一个按键检测标志 `keyen`，其由关键字 `static` 修饰，相当于一个全局变量，每次修改这个变量的时候其保留的都是上一次的值。引入一个按键检测标志是为了消除按键是否一直按下的情况，如果按键一直被按下 `keyen` 的值会一直在 `keyen` 的初始值和 `keyen-1` 之间循环，永远不会等于 0，则永远都不会被认为按键按下，需要等待释放。有关函数更加详细的说明看程序的注释即可。

主函数

代码清单 34-9 主函数

```
1 #include "stm32f4xx.h"
2 #include "../Bsp/beep/bsp_beep.h"
3 #include "../Bsp/usart/bsp_debug_usart.h"
4 #include "../Bsp/systick/bsp_SysTick.h"
5 #include "../Bsp/TouchPad/bsp_touchpad.h"
6
7 /**
8  * @brief 主函数
9  * @param 无
10  * @retval 无
11  */
12 int main(void)
13 {
14     uint16_t temp;
15     /* 初始化蜂鸣器 */
16     BEEP_GPIO_Config();
17
18     /* 初始化调试串口 */
19     Debug_USART_Config();
20     printf ( "\r\n 野火 STM32 输入捕获电容按键检测实验\r\n" );
21     printf ( "\r\n 触摸电容按键，蜂鸣器则会响\r\n" );
22
23
24     /* 初始化系统滴答定时器 */
25     //SysTick_Init();
26
27 // /* 这段注释的代码用于获取电容的充电时间（空载跟手按下的都可以测） */
28 // TPAD_TIM_Init();
29 // while(1)
30 // {
31 //     temp = TPAD_Get_Val();
32 //     printf("电容按键默认充电时间为: %d us\n",temp);
33 //     SysTick_Delay_Ms(500);
34 // }
35
36 // 初始化电容按键
37 while ( TPAD_Init() );
38
39 while (1) {
40     if ( TPAD_Scan() == TPAD_ON ) {
41         BEEP_ON;
42         //Delay_ms(25);
43         SysTick_Delay_Ms(25);
44         BEEP_OFF;
45     }
```

```
46     }  
47  
48 }
```

主函数初始化了蜂鸣器和串口，然后等待电容按键初始化成功，如果不成功则会一直等待。初始化成功之后，在一个 `while` 无限循环中不断的扫描按键，当按键按下之后蜂鸣器响 25ms，然后关掉

34.2.3 下载验证

把编译好的程序烧写到开发板，手按电容按键，蜂鸣器则会响。